

ShopSphere: Designing a Scalable Solution and Secure Database Architecture

Brady Finnell

CSCI360: Introduction to Database Systems

Dr. Tracy Lanham

Due: 24 Sep 2025

ABSTRACT

To effectively construct the ShopSphere database for analytics and efficient retrieval, a snowflake architecture is used. We connected all our tables through carefully thought-out and strategic foreign keys and primary keys. To enhance the performance of our data retrieval, we indexed columns based on their usage frequency, whether for updates or analytical purposes. Our fact and dimensions table structure allows us to distribute our database as needed. We manage the concurrency of replicated data among server locations using commit on a per-query basis. The ability to perform actions will be locked behind the level of authorization, authentication, and accounting.

ShopSphere is an e-commerce platform startup looking to revolutionize the way consumers find, view, research, and purchase products. Behind this is a team of engineers building a database, data flow framework, and data security framework to enable decision makers to enhance the user experience. The snowflake schema enables more efficient analytics without the cost of speed and performance in system logic. By the end, we aim to achieve a website and database that is efficient, scalable, maintains data integrity, and keeps the users' data safe in the process. We will cover measures taken for concurrency control, data integrity, data warehouse implementation, web platform integration, security measures, and scalability in a distributed environment.

Concurrency controls prevent the loss of data when one or more records are being updated at the same time. To prevent our customers and employees from this we implemented row-level locking within PostgreSQL with “SELECT...FOR UPDATE”. This will select the record and prevent access to the record until a commit or ROLLBACK within the transaction. Using optimistic concurrency with timestamps or versioning may be unreliable do to the

potential for vast amounts of customers to be performing transactions at the same time. Using data integrity, we can ensure those users maintain clean data. To accomplish this, primary and foreign keys authenticate the relationships between each of the tables in the database.

PostgreSQL has check functions integrated into the default library for these keys, providing integrity enforcement.

To permit data analyses to run on ShopSphere without imposing large demand on the server resources, a separate data warehouse is built with a snowflake schema. An Extract, Transform, Load (ETL) sequence is used to mirror fact and dimension tables from the transactional database. Normalization of the data formats by removal of redundancy is done by the transformation step and the usage of surrogate keys to allow consistency over time for dimension tables. We compute other analytical measures as well at the same step, e.g., sales total by order and repeat customer detection by a Boolean flag. Inserting into warehouse tables the nicely formatted and cleansed data after the correct sequence of dependency such that the dimension tables precede the facts is the last step. This process permits near real-time analytics without imposing too great a demand on the live transactional system even when usage of aggregate queries is heaviest.

Database ShopSphere communicates with the e-commerce website using a secure API kept with a secret key to maintain persistent safe communication between the website and database. Backend website services on the website take the incoming HTTP requests from the frontend and turn them into SQL queries or execution of stored procedures. Everything happens through parameterized queries, thus protecting the website from SQL attacks. When the customer walks through the option among the products or orders an order, the API reads from the respective dimension and fact tables, commits transactions, and updates stock levels when

required. This modular architecture enables the database schema to be modified, having little to no effect on the frontend due to the API providing a steady interface. In addition, caching technologies come into play on the high-traffic endpoints, like the product search, to reduce the query load on the database.

Through the employment of permission within our secure trio, we implement role-based access controls used to prevent anyone except those assigned particular roles — e.g., system administrators, analytical groups, or API services — from taking particular action. Sensitive customer information and passwords are secured by hashing algorithms applied on them, and the transactions' paths from database to application are encrypted through the employment of SSL/TLS encryption. Scheduled periodic backups, accompanied by off-site replication, are conducted to minimize the chance of loss of information due to hardware failure or a catastrophic event. Comprehensive audit trails on the whole administrative action are kept to enable the tracing of any information modified. As a group, the controls offer user confidence and ensure compliance with any standard applicable, e.g., the GDPR and CCPA.

To meet potential increases in the demands of traffic and information, the ShopSphere database is built for distributed deployment. The addition of replicating reads increases the availability for analytics and customer requests, freeing the master node to focus on processing transactions. We use partitioning techniques on large tables, such as structuring by date or geological region, enabling queries to run on smaller, focused datasets. Load balancing on the application server level balances web traffic evenly across the database, and the database tier offers possibilities for horizontal scaling consistent with user demand. By combining these strategies, ShopSphere is able to maintain high levels of efficiency and availability despite its growing customer base and the release of new products.

Through the implementation of a judiciously designed framework, effective management of concurrency, thorough ETL processes, safe web platform integration, and forward-looking thought toward security and scalability inherent within the design phase, the ShopSphere database is ideally suited to both current requirements of operation and future requirements of analytics. Furthermore, those considerations may be readily adopted by inexperienced database and SQL designers alike, consequently ensuring the system both properly supports current requirements but is also ideally positioned to support expansion, greater traffic, and shifting specifications commonplace within an e-commerce environment.

